

DATABASE

laboratory work 3

DML

Task 1: Generate and insert 200 random rows in your airport database.

To randomly generate 200 random rows, we mainly use:

```
INSERT INTO _ SELECT  
FROM generate_series(1, 200) AS i;
```

```
1 INSERT INTO Airline (airline_code, airline_name, airline_country, created_at, updated_at)
2 SELECT
3   'AIR' || i,
4   'Airline ' || i,
5   (ARRAY['Kazakhstan', 'USA', 'UK', 'UAE', 'Germany', 'France', 'Turkey'])[floor(random() * 7 + 1)],
6   NOW(),
7   NOW()
8 FROM generate_series(1, 200) AS i;
```

airline_id	airline_code	airline_name	airline_country	created_at	updated_at
190	AIR190	Airline 190	USA	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
191	AIR191	Airline 191	Germany	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
192	AIR192	Airline 192	UK	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
193	AIR193	Airline 193	UAE	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
194	AIR194	Airline 194	USA	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
195	AIR195	Airline 195	USA	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
196	AIR196	Airline 196	France	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
197	AIR197	Airline 197	Kazakhstan	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
198	AIR198	Airline 198	Turkey	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
199	AIR199	Airline 199	France	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
200	AIR200	Airline 200	France	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197

Total rows: 200 Query complete 00:00:00.143 CRLF Ln 8, C

We set i as random number from 1 – 200, and using this random number we set data for columns. Additionally, we also use (ARRAY[])[floor(random()) to pick random value from array. We also can use CASE _ END structure if columns are logically connected, for example country and city are connected, Paris cannot be located in US. So by sub request we can make if/else structure to connect them.

```
1 CASE city
2 WHEN 'Almaty' THEN 'Kazakhstan'
3 WHEN 'Astana' THEN 'Kazakhstan'
4 WHEN 'London' THEN 'UK'
5 WHEN 'New York' THEN 'USA'
6 WHEN 'Dubai' THEN 'UAE'
7 WHEN 'Paris' THEN 'France'
8 WHEN 'Istanbul' THEN 'Turkey'
9 END,
10 State || i,
11 city,
12 NOW(),
13 NOW()
14 FROM (
15 SELECT
16   i, (ARRAY['Almaty', 'Astana', 'London', 'New York', 'Dubai', 'Paris', 'Istanbul'])[floor(random() * 7 + 1)] AS city
17 FROM generate_series(1, 200) AS i
18 ) AS airport;
```

airport_id	airport_name	country	state	city	created_at	updated_at
186	Airport 186	France	State 186	Paris	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
187	Airport 187	Turkey	State 187	Istanbul	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
188	Airport 188	USA	State 188	New York	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
189	Airport 189	Turkey	State 189	Istanbul	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
190	Airport 190	Turkey	State 190	Istanbul	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
191	Airport 191	UK	State 191	London	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
192	Airport 192	USA	State 192	New York	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
193	Airport 193	France	State 193	Paris	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
194	Airport 194	Kazakhstan	State 194	Astana	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
195	Airport 195	Turkey	State 195	Istanbul	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
196	Airport 196	France	State 196	Paris	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
197	Airport 197	Kazakhstan	State 197	Astana	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
198	Airport 198	UK	State 198	London	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
199	Airport 199	France	State 199	Paris	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
200	Airport 200	Kazakhstan	State 200	Almaty	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197

Showing rows: 1 to 200 Page No: 1 of 1 CRLF Ln 31, Col 14

And we repeat this process for every table. For datetime: '1990-01-01'::DATE + (random() * 13000)::INT, this structure gives date from 1990 to 2026, random creates number from 0.0 to 1.0 by multiplying it by 13000 to we get days from 0 to 36 years.

Task 2: Add a new airline named "KazAir" based in "Kazakhstan" to the airline table.

```
INSERT INTO Airline (airline_code, airline_name, airline_country, created_at)
VALUES ('AIR201', 'KazAir', 'Kazakhstan', NOW());
```

Query

Query History

1

INSERT INTO Airline (airline_code, airline_name, airline_country, created_at)

2

VALUES ('AIR201', 'KazAir', 'Kazakhstan', NOW());

Data Output

Messages

Notifications

</

Task 3: Update the airline country "KazAir" to "Turkey".

```
UPDATE Airline SET airline_country = 'Turkey', updated_at = NOW()
WHERE airline_name = 'KazAir';
```

Query

Query History

1

UPDATE Airline

2

SET airline_country = 'Turkey',

3

updated_at = NOW()

4

WHERE airline_name = 'KazAir';

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 201

airline_id	airline_code	airline_name	airline_country	created_at	updated_at
[PK] integer	character varying (30)	character varying (50)	character varying (50)	timestamp without time zone	timestamp without time zone
186	AIR186	Airline 186	USA	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
187	AIR187	Airline 187	Turkey	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
188	AIR188	Airline 188	France	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
189	AIR189	Airline 189	Germany	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
190	AIR190	Airline 190	USA	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
191	AIR191	Airline 191	Germany	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
192	AIR192	Airline 192	UK	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
193	AIR193	Airline 193	UAE	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
194	AIR194	Airline 194	USA	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
195	AIR195	Airline 195	USA	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
196	AIR196	Airline 196	France	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
197	AIR197	Airline 197	Kazakhstan	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
198	AIR198	Airline 198	Turkey	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
199	AIR199	Airline 199	France	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
200	AIR200	Airline 200	France	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
201	AIR201	KazAir	Turkey	2026-09-27 08:13:01.28314	2026-09-27 08:21:08.329691

Total rows: 201

Query complete 00:00:00.083

Task 4: Add three airlines at once: "AirEasy" in "France", "FlyHigh" in "Brazil" and "FlyFly" in "Poland".

```
INSERT INTO Airline (airline_code, airline_name, airline_country, created_at) VALUES ('AIR202', 'AirEasy', 'France', NOW()), ('AIR203', 'FlyHigh', 'Brazil', NOW()), ('AIR204', 'FlyFly', 'Poland', NOW());
```

The screenshot shows a SQL query window with the following SQL code:

```
1 INSERT INTO Airline (airline_code, airline_name, airline_country, created_at)
2 VALUES
3 ('AIR202', 'AirEasy', 'France', NOW()),
4 ('AIR203', 'FlyHigh', 'Brazil', NOW()),
5 ('AIR204', 'FlyFly', 'Poland', NOW());
```

Below the query window, the 'Data Output' tab displays a table with 204 rows. The table columns are: airline_id, airline_code, airline_name, airline_country, created_at, and updated_at. The rows show existing airlines (AIR190 to AIR199) and the newly added ones (AIR200 to AIR204).

airline_id	airline_code	airline_name	airline_country	created_at	updated_at
190	AIR190	Airline 190	USA	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
191	AIR191	Airline 191	Germany	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
192	AIR192	Airline 192	UK	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
193	AIR193	Airline 193	UAE	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
194	AIR194	Airline 194	USA	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
195	AIR195	Airline 195	USA	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
196	AIR196	Airline 196	France	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
197	AIR197	Airline 197	Kazakhstan	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
198	AIR198	Airline 198	Turkey	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
199	AIR199	Airline 199	France	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
200	AIR200	Airline 200	France	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
201	AIR201	KazAir	Turkey	2026-09-27 08:13:01.28314	2026-09-27 08:21:08.329691
202	AIR202	AirEasy	France	2026-09-27 08:22:22.444685	[null]
203	AIR203	FlyHigh	Brazil	2026-09-27 08:22:22.444685	[null]
204	AIR204	FlyFly	Poland	2026-09-27 08:22:22.444685	[null]

Total rows: 204 Query complete 00:00:00.084

Task 5: Delete all flights whose arrival in 2024 year.

```
DELETE FROM Booking_flight WHERE flight_id IN (
    SELECT flight_id FROM Flights WHERE EXTRACT(YEAR FROM sch_arrival_time) = 2026 );
DELETE FROM Flights WHERE EXTRACT(YEAR FROM sch_arrival_time) = 2026;
```

Before: I used 2026 because I created arrival time as Now(), so every arrival was from 2026

The screenshot shows a SQL query window with the following SQL code:

```
1 DELETE FROM Booking_flight
2 WHERE flight_id IN (SELECT flight_id FROM Flights WHERE EXTRACT(YEAR FROM sch_arrival_time) = 2026);
3
4 DELETE FROM Flights
5 WHERE EXTRACT(YEAR FROM sch_arrival_time) = 2026;
```

Below the query window, the 'Data Output' tab displays a table with 200 rows. The table columns are: flight_id, sch_departure_time, sch_arrival_time, departing_airport_id, arriving_airport_id, departing_gate, arriving_gate, airline_id, and act_departure_time. The rows show flights scheduled for 2026.

flight_id	sch_departure_time	sch_arrival_time	departing_airport_id	arriving_airport_id	departing_gate	arriving_gate	airline_id	act_departure_time
100	2026-10-01 12:48:28.807197	2026-10-01 15:48:28.807197	109	163	Gate-2	Gate-28	3	2026-10-01 12:48:28.807197
101	2026-10-01 13:48:28.807197	2026-10-01 16:48:28.807197	129	119	Gate-1	Gate-1	11	2026-10-01 13:48:28.807197
102	2026-10-01 14:48:28.807197	2026-10-01 17:48:28.807197	124	142	Gate-27	Gate-16	53	2026-10-01 14:48:28.807197
103	2026-10-01 15:48:28.807197	2026-10-01 18:48:28.807197	27	7	Gate-3	Gate-22	174	2026-10-01 15:48:28.807197
104	2026-10-01 16:48:28.807197	2026-10-01 19:48:28.807197	136	175	Gate-30	Gate-19	26	2026-10-01 16:48:28.807197
105	2026-10-01 17:48:28.807197	2026-10-01 20:48:28.807197	95	80	Gate-1	Gate-23	46	2026-10-01 17:48:28.807197
106	2026-10-01 18:48:28.807197	2026-10-01 21:48:28.807197	187	53	Gate-5	Gate-17	56	2026-10-01 18:48:28.807197
107	2026-10-01 19:48:28.807197	2026-10-01 22:48:28.807197	109	40	Gate-11	Gate-8	78	2026-10-01 19:48:28.807197
108	2026-10-01 20:48:28.807197	2026-10-01 23:48:28.807197	142	54	Gate-28	Gate-30	182	2026-10-01 20:48:28.807197
109	2026-10-01 21:48:28.807197	2026-10-02 00:48:28.807197	136	14	Gate-14	Gate-19	138	2026-10-01 21:48:28.807197
110	2026-10-01 22:48:28.807197	2026-10-02 01:48:28.807197	2	199	Gate-21	Gate-28	185	2026-10-01 22:48:28.807197
111	2026-10-01 23:48:28.807197	2026-10-02 02:48:28.807197	145	174	Gate-6	Gate-3	87	2026-10-01 23:48:28.807197
112	2026-10-02 00:48:28.807197	2026-10-02 03:48:28.807197	50	84	Gate-17	Gate-3	159	2026-10-02 00:48:28.807197
113	2026-10-02 01:48:28.807197	2026-10-02 04:48:28.807197	164	65	Gate-18	Gate-30	167	2026-10-02 01:48:28.807197

Total rows: 200 Query complete 00:00:00.159

After: because every had 2026 every row got deleted

The screenshot shows a SQL query window with the following SQL code:

```
1 SELECT * FROM Flights
```

Below the query window, the 'Data Output' tab displays a table with 10 columns: flight_id, sch_departure_time, sch_arrival_time, departing_airport_id, arriving_airport_id, departing_gate, arriving_gate, airline_id, and act_departure_time. The table is empty, indicating that all flights scheduled for 2026 have been deleted.

flight_id	sch_departure_time	sch_arrival_time	departing_airport_id	arriving_airport_id	departing_gate	arriving_gate	airline_id	act_departure_time
-----------	--------------------	------------------	----------------------	---------------------	----------------	---------------	------------	--------------------

Task 6: Increase the price of all tickets in booking table for flights by 15%.

UPDATE Booking SET ticket_price = ticket_price * 1.15, updated_at = NOW();

Before:

Query

Query History

1

UPDATE Booking

2

SET ticket_price = ticket_price * 1.15,

3

updated_at = NOW();

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 200

Page No: 1

	booking_id [PK] Integer	flight_id integer	passenger_id integer	booking_platform character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone	status character varying (50)	ticket_price numeric (7,2)
1	1	16	1	Kaspi Travel	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	Confirmed	704.95
2	2	150	2	Agency	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	Pending	276.33
3	3	188	3	Website	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	Confirmed	587.33
4	4	177	4	Website	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	Cancelled	621.85
5	5	6	5	Mobile App	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	Confirmed	475.87
6	6	102	6	Mobile App	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	Cancelled	783.59
7	7	183	7	Website	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	Cancelled	449.32
8	8	89	8	Website	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	Confirmed	854.47
9	9	143	9	Agency	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	Pending	649.91
10	10	192	10	Website	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	Pending	158.23
11	11	175	11	Mobile App	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	Confirmed	437.84
12	12	5	12	Kaspi Travel	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	Cancelled	577.49
13	13	97	13	Kaspi Travel	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	Cancelled	844.83
14	14	138	14	Website	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	Pending	471.99
15	15	171	15	Kaspi Travel	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	Cancelled	440.30
Total rows: 200		Query complete 00:00:00.102						

After:

Query

Query History

1

SELECT * FROM booking

Data Output

Messages

Notifications

</

Task 7: Delete all tickets where price is less than 10000.

DELETE FROM booking_id IN (SELECT booking_id FROM
WHERE ticket_price
DELETE FROM booking_id IN (SELECT booking_id FROM WHERE
ticket_price
DELETE FROM booking_id IN (SELECT booking_id FROM
WHERE ticket_price
DELETE FROM booking_id IN (SELECT booking_id FROM WHERE
ticket_price
DELETE FROM WHERE ticket_price

We need to also delete all datas from tables that dependent on booking. Also, I used 750 because when I created ticket price I set it as 100 – 1000\$.

Query

Query History

1

SELECT * FROM booking

Data Output

Messages

Notifications

Task 8: Set a default code 'UNK' (Unknown) for any airline in the airline table where airline_code is missing (NULL).

```
UPDATE      SET airline_code 'UNK',
            updated_at
WHERE airline_code IS NULL
```

Query

Query History

1

UPDATE Airline

2

SET airline_code = 'UNK',

3

updated_at = NOW()

4

WHERE airline_code IS NULL;

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 204

	airline_id [PK] integer	airline_code character varying (30)	airline_name character varying (50)	airline_country character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	1	AIR1	Airline 1	UK	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
2	2	AIR2	Airline 2	France	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
3	3	null	Airline 3	Kazakhstan	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
4	4	AIR4	Airline 4	UAE	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
5	5	AIR5	Airline 5	Kazakhstan	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
6	6	null	Airline 6	Turkey	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
7	7	AIR7	Airline 7	UAE	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
8	8	AIR8	Airline 8	USA	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
9	9	null	Airline 9	Turkey	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
10	10	AIR10	Airline 10	Germany	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
11	11	AIR11	Airline 11	Kazakhstan	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
12	12	AIR12	Airline 12	France	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
13	13	AIR13	Airline 13	France	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
Total rows: 204		Query complete 00:00:00.115		Changes staged: Updated: 3		

Query Query History

1 SELECT * FROM airline
2

Data Output Messages Notifications

SQL

Showing rows: 1 to 204

	airline_id [PK] Integer	airline_code character varying (30)	airline_name character varying (50)	airline_country character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	1	AIR1	Airline 1	UK	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
2	2	AIR2	Airline 2	France	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
3	3	UNK	Airline 3	Kazakhstan	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
4	4	AIR4	Airline 4	UAE	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
5	5	AIR5	Airline 5	Kazakhstan	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
6	6	UNK	Airline 6	Turkey	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
7	7	AIR7	Airline 7	UAE	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
8	8	AIR8	Airline 8	USA	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
9	9	UNK	Airline 9	Turkey	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
10	10	AIR10	Airline 10	Germany	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
11	11	AIR11	Airline 11	Kazakhstan	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
12	12	AIR12	Airline 12	France	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
13	13	AIR13	Airline 13	France	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197

Total rows: 204 Query complete: 00:00:00.001 Changes staged: Updated: 2

Task 9: Delete all baggage check records created before June 1, 2023, whose result is still 'Not checked'.

DELETE FROM Baggage_check

Before:

Query

Query History

1

SELECT * FROM baggage_check

</

After: Just to see any results I modified input:

Query

Query History

1

DELETE FROM Baggage_check

2

WHERE created_at < '2026-09-28'

3

AND check_result = 'Flagged';

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 4

	baggage_check_id [PK] Integer	check_result character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone	booking_id integer	passenger_id integer
29	133	Failed	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	133	133
30	164	Cleared	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	164	164
31	166	Cleared	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	166	166
32	169	Passed	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	169	169
33	173	Passed	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	173	173
34	175	Passed	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	175	175
35	177	Cleared	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	177	177
36	182	Cleared	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	182	182
37	189	Passed	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	189	189
38	192	Passed	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	192	192
39	193	Passed	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	193	193
40	198	Cleared	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	198	198
41	200	Cleared	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197	200	200

Task 10: Delete all entries from the airport table where state is missing (NULL) and the city is either 'Mlawe' or 'Kepuh'.

DELETE FROM Airport

WHERE a_state IS NULL
AND IN 'Paris', 'Istanbul'

Query

Query History

1DELETE FROM Airport

2WHERE a_state IS NULL

3AND city IN ('Paris', 'Istanbul');

Data Output

Messages

Notifications

Task 11: Insert a new unchecked baggage inspection record and immediately return its newly generated baggage_check_id and created_at timestamp.

```
INSERT INTO Baggage_check (check_result, created_at, updated_at, booking_id,
passenger_id)
VALUES (
  'Not checked',
  NOW(),
  NOW(),
  1, 1
)RETURNING baggage_check_id, created_at;
```

Query

Query History

```
1 INSERT INTO Baggage_check (check_result, created_at, updated_at, booking_id, passenger_id)
2 VALUES (
3     'Not checked',
4     NOW(),
5     NOW(),
6     1,
7     1
8 )
9 RETURNING baggage_check_id, created_at, updated_at;
```

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 1

	baggage_check_id [PK] integer	created_at timestamp without time zone	updated_at timestamp without time zone
1	202	2026-09-27 10:33:10.652008	2026-09-27 10:33:10.652008

Task 12: Convert all country names in the airline table to uppercase to ensure data consistency.

```
SET airline_country = UPPER(airline_country),
    updated_at
```

Query

Query History

```

1 UPDATE Airline
2 SET airline_country = UPPER(airline_country),
3   updated_at = NOW();

```

Data Output

Messages

Notifications

+

📄

▼

📄

▼

🗑️

📄

📄

📄

SQL

Showing rows: 1 to 204

	airline_id [PK] integer	airline_code character varying (30)	airline_name character varying (50)	airline_country character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	1	AIR1	Airline 1	UK	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
2	2	AIR2	Airline 2	FRANCE	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
3	3	AIR3	Airline 3	KAZAKHSTAN	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
4	4	AIR4	Airline 4	UAE	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
5	5	AIR5	Airline 5	KAZAKHSTAN	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
6	6	AIR6	Airline 6	TURKEY	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
7	7	AIR7	Airline 7	UAE	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
8	8	AIR8	Airline 8	USA	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
9	9	AIR9	Airline 9	TURKEY	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
10	10	AIR10	Airline 10	GERMANY	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
11	11	AIR11	Airline 11	KAZAKHSTAN	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
12	12	AIR12	Airline 12	FRANCE	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048

Query

Query History

```

1 UPDATE Airline
2 SET airline_country = UPPER(airline_country),
3     updated_at = NOW();

```

Data Output

Messages

Notifications

+

📄

▼

📄

▼

🗑️

📄

📄

⬇️

📈

SQL

Showing rows: 1 to 204

🔍

📄

	airline_id [PK] integer	airline_code character varying (30)	airline_name character varying (50)	airline_country character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	1	AIR1	Airline 1	UK	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
2	2	AIR2	Airline 2	FRANCE	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
3	3	AIR3	Airline 3	KAZAKHSTAN	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
4	4	AIR4	Airline 4	UAE	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
5	5	AIR5	Airline 5	KAZAKHSTAN	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
6	6	AIR6	Airline 6	TURKEY	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
7	7	AIR7	Airline 7	UAE	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
8	8	AIR8	Airline 8	USA	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
9	9	AIR9	Airline 9	TURKEY	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
10	10	AIR10	Airline 10	GERMANY	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
11	11	AIR11	Airline 11	KAZAKHSTAN	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
12	12	AIR12	Airline 12	FRANCE	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048

Task 13: Update both the name and country for the airline with airline_id = 5. Set the name to 'Global Airways' and the country to 'United Kingdom', while updating update_at to the current date.

```

UPDATE Airline
SET airline_name = 'Global Airways',
    airline_country = 'United Kingdom',
    updated_at = NOW()
WHERE airline_id = 5;

```

Before:

Query

Query History

1

SELECT * FROM airline

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 204

	airline_id [PK] integer	airline_code character varying (30)	airline_name character varying (50)	airline_country character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	1	AIR1	Airline 1	UK	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
2	2	AIR2	Airline 2	FRANCE	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
3	3	AIR3	Airline 3	KAZAKHSTAN	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
4	4	AIR4	Airline 4	UAE	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
5	5	AIR5	Airline 5	KAZAKHSTAN	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
6	6	AIR6	Airline 6	TURKEY	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
7	7	AIR7	Airline 7	UAE	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
8	8	AIR8	Airline 8	USA	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
9	9	AIR9	Airline 9	TURKEY	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
10	10	AIR10	Airline 10	GERMANY	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
11	11	AIR11	Airline 11	KAZAKHSTAN	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048

After:

Query

Query History

1

UPDATE Airline

2

SET airline_name = 'Global Airways',

3

airline_country = 'United Kingdom',

4

updated_at = NOW()

5

WHERE airline_id = 5;

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 204

	airline_id [PK] integer	airline_code character varying (30)	airline_name character varying (50)	airline_country character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	5	AIR5	Global Airways	United Kingdom	2026-09-27 07:48:28.807197	2026-09-27 10:37:39.05996
2	1	AIR1	Airline 1	UK	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
3	2	AIR2	Airline 2	FRANCE	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
4	3	AIR3	Airline 3	KAZAKHSTAN	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
5	4	AIR4	Airline 4	UAE	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
6	6	AIR6	Airline 6	TURKEY	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
7	7	AIR7	Airline 7	UAE	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
8	8	AIR8	Airline 8	USA	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
9	9	AIR9	Airline 9	TURKEY	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
10	10	AIR10	Airline 10	GERMANY	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048
11	11	AIR11	Airline 11	KAZAKHSTAN	2026-09-27 07:48:28.807197	2026-09-27 10:35:25.726048

Task 14: Set the state field to 'Capital District' for airports located in specific cities: 'Astana', 'London', and 'Tokyo'

```

UPDATE Airport
SET a_state = 'Capital District',

```

updated_at
WHERE city IN 'Astana', 'London', 'Tokyo'
Before:

Query

Query History

1

```
SELECT * FROM airport
```

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 197

Page No: 1

	airport_id [PK] integer	airport_name character varying (50)	country character varying (50)	a_state character varying (50)	city character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone
1	1	Airport 1	USA	State 1	New York	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
2	2	Airport 2	UK	State 2	London	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
3	3	Airport 3	Kazakhstan	State 3	Almaty	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
4	4	Airport 4	UAE	State 4	Dubai	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
5	5	Airport 5	Kazakhstan	State 5	Almaty	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
6	6	Airport 6	Kazakhstan	State 6	Astana	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
7	7	Airport 7	UK	State 7	London	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
8	8	Airport 8	Turkey	State 8	Istanbul	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
9	9	Airport 9	UK	State 9	London	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
10	10	Airport 10	USA	State 10	New York	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
11	11	Airport 11	Kazakhstan	State 11	Almaty	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197

After:

Query

Query History

1

UPDATE Airport

2

SET a_state = 'Capital District',

3

updated_at = NOW()

4

WHERE city IN ('Astana', 'London', 'Tokyo');

Data Output

Messages

Notifications

+ ↩ ⏮ ⏭ 🗑️ 📄 ⬇️ ⚙️ SQL

Showing rows: 1 to 197

🔍

 Page No: 1

	airport_id integer PK integer 185	airport_name character varying (50) Airport 185	country character varying (50) USA	a_state character varying (50) State 185	city character varying (50) New York	created_at timestamp without time zone 2026-09-27 07:48:28.807197	updated_at timestamp without time zone 2026-09-27 07:48:28.807197
134							
135	189	Airport 189	Turkey	State 189	Istanbul	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
136	192	Airport 192	USA	State 192	New York	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
137	193	Airport 193	France	State 193	Paris	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
138	196	Airport 196	France	State 196	Paris	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
139	200	Airport 200	Kazakhstan	State 200	Almaty	2026-09-27 07:48:28.807197	2026-09-27 07:48:28.807197
140	2	Airport 2	UK	Capital District	London	2026-09-27 07:48:28.807197	2026-09-27 10:39:30.944769
141	6	Airport 6	Kazakhstan	Capital District	Astana	2026-09-27 07:48:28.807197	2026-09-27 10:39:30.944769
142	7	Airport 7	UK	Capital District	London	2026-09-27 07:48:28.807197	2026-09-27 10:39:30.944769
143	9	Airport 9	UK	Capital District	London	2026-09-27 07:48:28.807197	2026-09-27 10:39:30.944769
144	12	Airport 12	UK	Capital District	London	2026-09-27 07:48:28.807197	2026-09-27 10:39:30.944769
145	14	Airport 14	UK	Capital District	London	2026-09-27 07:48:28.807197	2026-09-27 10:39:30.944769
146	20	Airport 20	UK	Capital District	London	2026-09-27 07:48:28.807197	2026-09-27 10:39:30.944769

Task 15: Mark all baggage check records (baggage_check) created in March 2024 with a status of 'Not checked' as completed ('Checked')

```
UPDATE Baggage_check
SET check result = 'Checked';
```

updated_at

WHERE created_at '2024-03-01'

AND created_at '2024-04-01'

AND check_result 'Not checked'